ASSIGNMENT 4

Introduction to Asymmetric Cryptography

Cryptography and Network Security

Topic coverage: Public-key principles, symmetric vs asymmetric systems, public-key building blocks, history, hybrid cryptography, RSA algorithm and key generation, key sizes, and RSA security considerations.

Note: This document provides model answers to all eight questions, including labelled process diagrams, a comparison table, RSA formulas, a worked example, and a flowchart.

Q1. Overview and Principles of Asymmetric Cryptography [10 Marks]

Definition. Asymmetric cryptography, also called public-key cryptography, uses a mathematically related key pair: a **public key** that can be shared openly and a **private key** that must remain secret. What is protected with one key can be verified or, in suitable schemes, reversed only with the corresponding key. The exact operation depends on whether encryption, signatures, or key agreement is being used.

Roles of the keys

- **Public key:** Shared with other users. It may be used to encrypt data for the key owner, verify the owner's digital signature, or participate in a key-agreement protocol.
- **Private key:** Kept secret by its owner. It may decrypt data encrypted for that owner, create digital signatures, or contribute to key agreement. Private keys should be protected with access controls and secure storage.

Confidentiality: Alice encrypts a message using Bob's public encryption key. Only Bob, using the matching private key, should be able to decrypt it. **Authentication and integrity:** Alice signs a message or its hash with her private signing key; Bob verifies the signature with Alice's public key. A valid signature supports origin authentication and detects changes to the signed data. **Non-repudiation:** A signature can provide evidence that a particular private key signed the data, but this depends on trustworthy key ownership, private-key control, identity validation, and legal/operational context; it is not an absolute guarantee by mathematics alone.

Labelled process diagram

ALICE (sender)	Transmission	BOB (receiver)
Plaintext message M	Encrypt using Bob's public key: $C = E(PU_B, M)$	Ciphertext C
—	C ■■	Decrypt using Bob's private key: $M = D(PR_B, C)$

Signature path: Alice computes a hash of M and signs it with her private signing key. Bob verifies the signature with Alice's public key and independently hashes M. Matching verification results support integrity and authentication. Encryption and signatures are distinct operations; signing is not simply "encrypting with the private key."

Q2. Symmetric vs Asymmetric Cryptography [8 Marks]

Heading	Symmetric cryptography	Asymmetric cryptography
Number of keys	One shared secret key is used by both parties.	A public/private key pair is used.
Key distribution	The secret key must be delivered or established securely in advance.	Public keys may be distributed openly, but their authenticity must be verified. Private keys remain secret.
Speed	Generally very fast; suitable for bulk data.	Generally slower and computationally more expensive.
Typical key length	Common modern examples include AES-128 and AES-256.	Common examples include RSA-2048/3072 and ECC keys such as 256-bit curves; sizes are not directly comparable.
Scalability for n users	If every pair needs a separate key, n(n–1)/2 pairwise keys may be needed.	Each user can maintain one key pair; public-key distribution still needs a trusted method.
Typical applications	Disk/file encryption, VPN traffic, database encryption, and bulk TLS traffic.	Key establishment, digital signatures, certificates, and encrypting small secrets.

Summary: Symmetric cryptography is efficient for large amounts of data, while asymmetric cryptography helps solve key-establishment and identity-verification problems. Secure systems commonly combine both.

Q3. Main Techniques and Building Blocks [10 Marks]

1. Trapdoor one-way functions

A one-way function is easy to compute but computationally infeasible to reverse without special information. A trapdoor is secret information that makes inversion feasible. RSA is based on operations involving modular arithmetic where finding the private operation without the private key is believed to be infeasible for appropriately sized parameters. The security claim depends on mathematical assumptions and correct implementation.

2. Diffie–Hellman (DH) key exchange

DH allows two parties to derive a shared secret over a public channel without sending the secret itself. Public parameters include a suitable group and generator g. Alice chooses secret a and sends $A = g^a$; Bob chooses secret b and sends $B = g^b$. Alice computes B^a and Bob computes A^b; both obtain g^{ab}. Basic DH does not authenticate participants by itself, so an unauthenticated exchange can be vulnerable to a man-in-the-middle attack. Certificates or another authentication mechanism are needed.

3. Digital signatures

A signing algorithm uses the signer's private key to produce a signature over a message or its cryptographic hash. A verification algorithm uses the corresponding public key. Signatures provide integrity and evidence of possession/control of the signing key and can support authentication and non-repudiation when identity and key management are reliable. They do not, by themselves, provide confidentiality.

4. Public-Key Infrastructure (PKI) and certificates

PKI is a set of policies, roles, software, and procedures for creating, distributing, validating, renewing, and revoking public-key certificates. A certificate binds an identity (such as a website domain) to a public key and is digitally signed by a Certificate Authority (CA). A client validates the certificate chain, name, validity period, and applicable status/policy checks before trusting the key. PKI helps prevent an attacker from substituting their own public key.

How they work together: a certificate can authenticate a server's public key; an authenticated key exchange can establish a shared session secret; symmetric authenticated encryption can then protect application data efficiently. Digital signatures can authenticate software, documents, or protocol messages.

Q4. Historical Development of Public-Key Cryptography [10 Marks]

(a) Key-distribution problem. Traditional symmetric cryptography requires communicating parties to share a secret key. Securely distributing and refreshing keys becomes difficult as the number of users grows, particularly when parties have never met. Public-key cryptography was developed to address this problem and enable public verification of signatures.

(b) Diffie and Hellman, 1976. Whitfield Diffie and Martin Hellman published "New Directions in Cryptography," introducing the public-key concept and describing a method for key agreement over an insecure channel. The Diffie–Hellman key exchange lets two parties derive a shared secret without transmitting that secret directly. Key agreement alone does not authenticate the parties.

(c) RSA, 1977. Ronald Rivest, Adi Shamir, and Leonard Adleman developed RSA, a public-key cryptosystem based on modular arithmetic and the practical difficulty of factoring a large composite modulus. RSA can be used for encryption in suitable constructions and for digital signatures. Modern use requires carefully designed padding and protocols; textbook RSA is not secure on its own.

(d) Earlier classified work at GCHQ. British researchers James Ellis, Clifford Cocks, and Malcolm Williamson independently developed related ideas within the UK Government Communications Headquarters (GCHQ) during the 1960s and early 1970s. Ellis described a non-secret encryption concept, Cocks developed an RSA-like system, and Williamson developed a key-agreement method related to Diffie–Hellman. This work remained classified for years and became public later.

(e) ElGamal and ECC. Taher ElGamal introduced a public-key encryption/signature approach in the 1980s based on the discrete logarithm problem. Elliptic Curve Cryptography (ECC) applies elliptic-curve group operations to key agreement and signatures, offering strong security with comparatively small keys. Examples include ECDH for key agreement and ECDSA/EdDSA for signatures. These systems rely on different mathematical assumptions from RSA.

Overall impact: public-key techniques made scalable key establishment, digital signatures, certificates, and secure internet protocols practical. Current systems still depend on sound algorithms, implementation, and key management.

Q5. Advantages, Disadvantages and Hybrid Cryptosystems [8 Marks]

Advantages

- Public keys can be distributed widely, reducing the need to pre-share a unique secret with every correspondent.
- Digital signatures support integrity checking, origin authentication, and evidence of signing-key use.
- Key-agreement protocols can establish session secrets over networks that are not physically secure.

Disadvantages

- Public-key operations are usually slower than symmetric encryption and are not ideal for bulk data.
- Public keys must be authenticated; otherwise, attackers can substitute their own keys.
- Private-key theft, weak parameters, implementation flaws, and poor certificate management can undermine security.

Why TLS is hybrid: asymmetric cryptography authenticates a server and/or establishes key material, while symmetric authenticated encryption protects the high-volume application traffic. This combines the convenient key establishment of public-key techniques with the speed of symmetric algorithms. In modern TLS, ephemeral (EC)DHE is commonly used to establish shared secrets, certificates/signatures authenticate the handshake, and a key derivation function produces traffic keys. AES-GCM or ChaCha20-Poly1305 can then protect records. RSA certificates may be used for signatures in some configurations, but modern TLS does not use textbook RSA encryption to transport the session key.

Example: when a browser connects to an HTTPS website, it validates the server certificate, authenticates the handshake, derives session keys, and then encrypts page data with symmetric authenticated encryption. The browser does not perform a costly public-key operation on every byte of the webpage.

Q6. RSA Algorithm and Key Generation [12 Marks]

1. Key generation

- **Step 1:** Choose two large, distinct random primes p and q.
- **Step 2:** Compute the modulus $n = p \times q$. The public and private operations use this modulus.
- **Step 3:** Compute Euler's totient $\phi(n) = (p - 1)(q - 1)$, when p and q are prime.
- **Step 4:** Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. A common choice is $e = 65537$.
- **Step 5:** Compute the private exponent d, the modular multiplicative inverse of e modulo $\phi(n)$, so that $d \times e \equiv 1 \pmod{\phi(n)}$.
- **Step 6:** Publish the public key (n, e); keep d and the prime factors secret. Implementations normally also protect p, q and any CRT parameters.

2. Encryption and decryption

Represent the message as an integer m in the valid range $0 \le m < n$ after applying a secure encoding/padding scheme. Textbook RSA mathematics is:

```
Encryption: c = m^e mod n
Decryption: m = c^d mod n
```

Worked toy example (for learning only): choose p = 61 and q = 53. Then n = 3233 and $\phi(n) = 3120$. Choose e = 17; its inverse modulo 3120 is d = 2753 because $17 \times 2753 = 46801 = 1 + 15 \times 3120$. For message m = 65, encryption gives $c = 65^{17} \bmod 3233 = 2790$; decryption gives $2790^{2753} \bmod 3233 = 65$. These tiny values are insecure and must never be used in real systems.

3. Why decryption recovers the message

Because $ed \equiv 1 \pmod{\phi(n)}$, there is an integer k such that $ed = 1 + k\phi(n)$. For messages in the valid RSA domain, Euler's theorem and the RSA construction imply $m^{ed} \equiv m \pmod{n}$; the result can be justified more generally using the congruences modulo p and q and the Chinese Remainder Theorem, including messages not coprime to n. Therefore $c^d \equiv m \pmod{n}$. The proof assumes correctly generated keys and the proper mathematical domain.

RSA flowchart

START
Generate random primes p, q
Compute n = pq and ϕ(n) = (p–1)(q–1)
Choose e; compute d = e■¹ mod ϕ(n)
Public key (n,e) | Private key (n,d)
Encode/pad message; encrypt c = m■ mod n
Decrypt m = c■ mod n; decode message
END

Practical caution: never implement raw textbook RSA for real data. Use standardized padding and a vetted cryptographic library. For encryption, RSA-OAEP is the established RSA encryption padding; for signatures, use a proper signature scheme such as RSA-PSS or a protocol-specified alternative.

Q7. RSA Key Sizes, ECC Equivalence and Quantum Threat [10 Marks]

Key size means the bit length of RSA modulus n, not the number of bits in e or d. RSA's security is closely related to the difficulty of factoring n into its secret prime factors p and q. If an attacker factors n, they can compute ϕ(n) and derive the private exponent d. Larger moduli make the best known classical factoring attacks much more expensive, but also increase computation and key size.

RSA modulus size	General interpretation	Approximate ECC equivalent strength
1024-bit	Obsolete/insecure for modern protection; do not deploy.	About 160-bit ECC (roughly 80-bit security), also obsolete.
2048-bit	Common minimum for many current uses; about 112-bit classical security.	About 224-bit ECC (roughly 112-bit security).
3072-bit	Often selected for about 128-bit classical security and longer-term margins.	About 256-bit ECC (roughly 128-bit security).
4096-bit	Higher classical factoring cost, but more computational overhead; does not automatically guarantee a longer useful lifetime.	No exact one-to-one mapping; approximately 256–384-bit ECC is commonly discussed for this range depending on the comparison and target strength.

ECC and RSA key lengths are not directly comparable: a 256-bit elliptic-curve key can provide security similar to a much larger RSA modulus. Exact estimates vary by standard, curve, and security target; consult current standards for a deployment decision.

Quantum-computing threat: Shor's algorithm, if run on a sufficiently large fault-tolerant quantum computer, could factor RSA moduli efficiently and solve the discrete-log problems used by conventional ECC. Such a machine would undermine RSA and conventional ECC public-key security. Current quantum hardware has not demonstrated this capability at practical cryptographic sizes, but long-lived confidential information may face "harvest now, decrypt later" risk. Organizations should plan migration to standardized post-quantum cryptography and support crypto agility.

Recommendation: do not use RSA-1024. RSA-2048 is a common baseline, while RSA-3072 may be chosen for a stronger classical security target. RSA-4096 has higher cost and is not a substitute for a post-quantum migration plan.

Q8. RSA Security Considerations and Attacks [12 Marks]

(a) Factoring attacks. If an attacker factors n = pq, the attacker can compute ϕ(n) and derive d. Use adequately sized moduli generated from strong random primes, avoid reused or poorly generated primes, and use maintained cryptographic libraries.

(b) Small public exponent and small-message attacks. A small exponent such as e = 3 can be unsafe in certain unpadded or improperly padded situations, especially when the message is small or the same message is exposed

under multiple keys. The exponent 65537 is widely used, but a safe exponent alone does not make RSA encryption secure. Use randomized OAEP padding and follow protocol requirements.

(c) Common modulus attack. If the same modulus n is reused with different public exponents for the same message under particular conditions, an attacker may recover the message using a combination of ciphertexts and number theory. Generate independent key pairs for separate users and applications; do not share RSA moduli across principals.

(d) Timing and other side-channel attacks. Differences in execution time, power use, cache activity, or electromagnetic emissions can leak information about private-key operations. Use constant-time implementations, RSA blinding where supported, hardened libraries, secure hardware where appropriate, and avoid exposing detailed error behavior.

(e) Chosen-ciphertext attacks. An attacker may submit crafted ciphertexts and infer information from the receiver's responses. Historically, distinguishable padding errors and protocol behavior have enabled attacks against poorly designed RSA encryption. RSA-OAEP, correctly implemented, is designed to provide chosen-ciphertext security under standard assumptions; systems must avoid padding oracles and follow uniform error-handling guidance.

Why OAEP padding is required. Raw RSA is deterministic and algebraically malleable: the same message yields the same ciphertext, and ciphertexts can be transformed in mathematically related ways. OAEP adds randomized encoding and is the standard padding construction for RSA encryption in many applications. Padding does not compensate for bad key generation or a vulnerable implementation.

RSA digital signatures. A secure signature scheme hashes the message with an approved hash function and applies a standardized signature encoding and private-key operation. The recipient verifies the signature using the public key. RSA-PSS is generally preferred for new RSA signature applications where supported. A signature is not encryption and does not hide the message.

Good practices for secure RSA deployment

- Use current, approved key sizes (at least RSA-2048 where RSA is required); never use RSA-1024 for new security deployments.
- Use RSA-OAEP for encryption and RSA-PSS for signatures where the relevant protocol supports them; never use textbook RSA.
- Generate keys with a cryptographically secure random-number generator and a maintained cryptographic library.
- Keep private keys encrypted and access-controlled; rotate, revoke, back up, and retire keys according to policy.
- Validate certificate chains, names, expiry dates, and revocation/status information as appropriate.
- Use constant-time and side-channel-hardened implementations; do not expose distinguishable decryption errors.
- Use RSA to protect small secrets or establish protocol security, not to encrypt large files directly. Use hybrid encryption for bulk data.
- Plan for post-quantum migration where data confidentiality must last into a future quantum-capable era.

Conclusion: RSA remains useful when deployed with appropriate parameters, padding, protocol design, and key management. Most serious failures arise not only from mathematical attacks but also from misuse, poor randomness, side channels, and weak surrounding systems.

References for further reading

1. NIST, *Recommendation for Key Management: Part 1 – General* (SP 800-57 Part 1).
2. NIST, *Digital Signature Standard (DSS)* (FIPS 186-5).
3. NIST, *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography* (SP 800-56A).
4. RFC 8017, *PKCS #1: RSA Cryptography Specifications Version 2.2*.
5. Diffie, W. and Hellman, M. E., "New Directions in Cryptography," IEEE Transactions on Information Theory, 1976.